PROJECT REVIEW / CLEANUP ROADMAP

A cleaner, lighter WEBXPAY project

An evidence-led plan for reducing weight, improving code boundaries, and making maintenance safer - without a risky rewrite.

Recommendation: keep the current Next.js App Router foundation. Improve it in small, measured batches, starting with assets and route-level client JavaScript.

25 MB	82.3%	24	12
current public assets	reduction in six optimized images	client-marked source files	page routes

CURRENT POSITION

The first measured asset pass is complete: six large image files were reduced from 9.52 MB to 1.69 MB, saving 7.47 MiB. The production build passed after the path updates. The next best return is to measure client JavaScript and safely consolidate confirmed duplicate assets.

Prepared 31 October 2026

Scope: current repository snapshot and verified work completed in this session. Existing cleanup/audit reports were not used as input and remain untouched.

01 / BASELINE

What the project is today

The codebase is a Next.js 15 / React 19 marketing site using the App Router. The current structure is already grouped by route and feature; the cleanup opportunity is targeted simplification, not replacement.

Area	Observed structure	Implication
Routes	12 App Router pages plus 2 API routes.	Route ownership is clear; preserve URLs and page behavior.
UI code	68 component files grouped into common, home, press, pricing, product, and solution areas.	Keep feature grouping; extract only proven shared patterns.
Assets	256 files under public; directory is about 25 MB after the image pass.	Largest removable weight is delivered media, not TypeScript source.
Client code	24 files include a client directive.	Audit why each needs browser state/effects; do not convert mechanically.
Local generated data	node_modules about 676 MB; .next about 241 MB after install/build.	Large checkout footprint, but generated folders are not production asset weight.

Current source boundaries

Path	Current responsibility	Direction
app/	Routes, layouts, API handlers, page-level data.	Keep routing and route metadata here; keep pages composed and readable.
components/common/	Header, footer, menu, shared feature rows.	Use for genuinely cross-route UI, not as a catch-all.
components//	Page sections and product-specific modules.	Retain these feature folders; standardize names gradually.
lib/	Small shared utilities and mail integration.	Add server-only helpers here when needed; avoid speculative layers.
public//	Static images, icons, sitemap.	Keep stable public paths unless all references are migrated and checked.

Build baseline: Next.js production build succeeded after the image-path updates. The reported first-load JavaScript is 102 kB shared across routes and 182 kB for the home route. Treat these as comparison baselines, not as proof that all pages need optimization.

Where cleanup can pay off

Ranked by likely value and risk. Estimates describe the measured snapshot; they are not promises of equal user-visible savings.

Priority	Finding	Opportunity / caution
P1 - media	Six large assets were optimized: 9.52 MB to 1.69 MB, 82.3% smaller. Public directory fell from about 33 MB to about 25 MB.	Continue with referenced images, fit-for-purpose dimensions, and visual review. Preserve transparent edges and responsive crops.
P1 - client JS	The root layout mounts the chatbot and Govpay globally; ChatbotWidget is a large client component.	Measure where each feature is required, then lazy-load or scope by route/interaction. Do not remove global behavior without product approval.
P2 - duplicate files	Seven exact byte-identical image pairs were found, around 1.46 MiB of duplicate storage.	Potential saving is disk/deploy payload only if references can share one URL. Check CSS, dynamic paths, and public URLs before consolidation.
P2 - code repetition	Several route families use similar banners and page sections.	Extract one shared component only where markup, behavior, and responsive rules truly match. Avoid one configurable mega-component.
P2 - dependencies	package.json declares Next 15.5.x while eslint-config-next is pinned to 15.3.1.	Align versions intentionally and test CI. Audit actual imports before removing dependencies.
P3 - deploy workflow	The EC2 workflow performs npm ci and a production build on the server.	Measure deploy duration and memory first; consider CI-built artifacts only if the measured operational benefit justifies the change.

Important distinction

The 676 MB node_modules and 241 MB .next folders are local generated/install output, not the public-site download size. Their size matters for developer disk use and build workflow, while public images and route JavaScript matter to visitors. Keep those measurements separate.

Dependency health note: npm ci reported 30 dependency audit findings (18 moderate, 10 high, 2 critical). This report does not assess exploitability and no automatic dependency fixes were applied. Triage separately before changing versions.

03 / TARGET

A simpler structure, without a rewrite

The goal is to make ownership obvious and reduce duplicate work while preserving route names, public URLs, and the existing design system.

Boundary	Keep / clarify	Avoid
app/ routes	One page entry per route; data loading and route metadata at the route boundary.	Putting unrelated shared UI, large data blobs, and browser behavior inside page entry files.
components/common/	Navigation, footer, truly shared controls, repeated site-wide patterns.	Using common/ as a home for components used by only one feature.
components/features/	Existing feature folders are a good base; group page sections by domain and name consistently.	Large path-only folder moves that create noisy diffs but no behavior or ownership improvement.
lib/	Small reusable helpers; separate server-only integrations from browser-safe utilities.	A generic framework of abstractions for one-off page content.
public/	Feature-named assets with consistent filenames; preserve URL compatibility.	Removing assets based only on filenames or a source search that misses dynamic/CSS references.

Practical conventions to adopt

- **One owner per feature:** keep a feature component, its content data, and its assets easy to locate together.
- **Keep components server-first:** add a client boundary only for browser state, effects, event handlers, or browser-only APIs.
- **Use shared UI by evidence:** consolidate repeated sections after confirming matching markup, semantics, and responsive behavior.
- **Keep paths boring:** descriptive, stable asset names; avoid renaming live public URLs unless every consumer is migrated.
- **Prefer small diffs:** one theme per cleanup batch, easy to review and revert.

Do not reorganize the whole repository. Its current feature-oriented layout is serviceable; the biggest wins are asset weight, client boundaries, and careful deduplication.

A phased, reversible cleanup

Sequence work so each step has a baseline, a clear acceptance check, and a small rollback surface.

0

Protect baseline

Use a clean branch/worktree for new cleanup; preserve current local edits. Record build output, routes, public asset sizes, and representative page screenshots.

Exit check: No existing work overwritten; before/after comparison is reproducible.

1

Finish media cleanup

Complete the verified image conversions already underway. Inventory remaining large assets and exact duplicates; consolidate only after references and visuals are checked.

Exit check: All paths resolve; representative desktop/mobile views look right; record bytes saved per route and overall.

2

Reduce client work

Measure route JS and locate browser-only behavior. Test route-scoping or on-interaction loading for chatbot and any nonessential global widgets.

Exit check: No unwanted UI regressions; no unrelated route bundle growth; compare home first-load baseline of 182 kB.

3

Clarify code ownership

Normalize names in touched areas, move feature-specific data out of component presentation where useful, and extract only proven repeated UI.

Exit check: Small readable diffs; unchanged URLs and page content; build, lint, and type checks pass.

4

Dependency / deploy hygiene

Review imports and package versions; align Next and its lint config. Measure CI and EC2 install/build time before changing deploy architecture.

Exit check: Lockfile reproducible; deploy rollback documented; dependency changes reviewed and tested.

Suggested order is value-first, not calendar-first. Stop after any phase if the measured gain is not worth the review and regression risk.

How to call cleanup successful

Success is less unnecessary weight and clearer ownership without changing what customers see or breaking deployments.

Measure	Baseline / test	Acceptance rule
Production build	npm run build passed after the image updates.	Must pass after each cleanup batch.
Types and lint	Build performed type validation and linting; standalone script is npm run lint.	Run project lint and TypeScript checks explicitly when changes warrant; resolve new failures, do not hide them.
Asset integrity	Six optimized images saved 7.47 MiB. Updated local references resolve.	No broken image paths; visual check on the page and relevant viewport; compare dimensions/alpha where applicable.
JavaScript	Build snapshot: shared 102 kB; home first load 182 kB.	Track route-level changes; prevent unrelated routes from growing without a documented reason.
Public URL compatibility	Assets are served directly from public/.	Keep existing URLs where possible; if changed, verify every static, CSS, data-driven, and external use.
Dependency/deploy stability	npm ci succeeded; audit reported findings. EC2 currently builds on server.	Lockfile install and deployment smoke test pass; review vulnerabilities and release rollback separately.

Risks and mitigations

- **Visual regressions:** preserve dimensions and alpha; use before/after screenshots for hero, mobile, and content imagery.
- **Hidden asset references:** search source, CSS, data, and generated/dynamic paths; verify network requests on affected routes.
- **Behavior changes from lazy-loading:** keep keyboard access, analytics, loading states, and mobile interactions in the acceptance checks.
- **Existing local edits:** the current checkout contains substantial pre-existing changes. Do not reset, mass-format, or mix unrelated cleanup into one commit.
- **Over-cleaning:** do not remove a dependency or component until actual use and indirect use are checked.

Next recommended batch: consolidate only the seven verified duplicate image pairs after reference checks, then measure route JavaScript before changing global client features.

Report basis

Direct inspection of the current repository structure, package manifest, deployment workflow, public asset inventory, exact duplicate hashes, and the successful production build after the image optimization pass. Existing cleanup/audit reports were deliberately excluded. Snapshot date: 31 October 2026.